

Soccer Injury Prediction Model - Detailed Code Explanation

This document explains every major block of the Google Colab notebook used to build a Soccer Player Injury Prediction Model. For each block, it explains:

- What the code does
- Why it is needed
- How it contributes to the machine learning pipeline
- Important concepts behind the code

1. Installing Libraries

The first code cell installs external Python packages such as XGBoost, scikit-learn, imbalanced-learn, pandas, NumPy, matplotlib, and seaborn. Why it is used: • Google Colab starts with a clean environment. • Some required packages may not already be installed. • XGBoost is the machine learning algorithm used. • imbalanced-learn provides SMOTE for class balancing. Without these libraries the notebook cannot run.

2. Import Statements

Import statements load tools into memory. Examples: • pandas → data handling. • numpy → numerical calculations. • matplotlib and seaborn → visualization. • sklearn → machine learning utilities. • XGBClassifier → XGBoost model. Why it is used: Imports allow the notebook to access functions that would otherwise be unavailable.

3. Uploading and Loading the Dataset

The dataset is uploaded and read using pandas.read_csv(). Why it is used: • Converts CSV data into a DataFrame. • Makes filtering, analysis, and machine learning possible. • Provides a structured table format. The DataFrame becomes the foundation of the entire project.

4. Creating the Target Variable

The target variable is: target_major_injury = 1 if season_days_injured > 120 target_major_injury = 0 otherwise Why it is used: Machine learning requires a target column that represents what we want to predict. This transforms a continuous variable (days injured) into a binary classification problem.

5. Removing Missing Targets

Rows with missing injury-day values are removed. Why it is used: The model cannot learn from rows where the correct answer is unknown. Keeping missing targets would introduce errors during training.

6. Automatic Feature Selection

The notebook automatically selects numeric columns while excluding: • p_id2 • season_days_injured • target_major_injury Why it is used: • IDs do not contain predictive information. • season_days_injured would leak the answer. • Numeric features can be directly used by XGBoost. This approach adapts automatically to your dataset.

7. Creating X and y

X = feature matrix y = target vector Why it is used: Machine learning models expect: • X → input variables • y → output labels This separation is a standard requirement for supervised learning.

8. Handling Missing Values

Missing feature values are replaced using the median. Why median? • Resistant to outliers. • More robust than mean. • Preserves realistic feature values. This ensures XGBoost receives complete data.

9. Train-Test Split

The dataset is divided into: • 70% Training • 30% Testing Why it is used: Training data teaches the model. Testing data evaluates performance on unseen examples. Stratify=y preserves the class distribution in both sets.

10. SMOTE Oversampling

SMOTE = Synthetic Minority Oversampling Technique. Problem: Major injuries are rare. Solution: SMOTE generates synthetic injury examples. Why it is used: Without balancing, the model would mostly predict 'No Injury'. SMOTE helps the model learn minority-class patterns.

11. XGBoost Model

XGBoost is an ensemble of gradient-boosted decision trees. Why it is used: • Excellent performance on tabular data. • Handles nonlinear relationships. • Robust to noisy data. • Works well with mixed feature scales. Key hyperparameters: • `n_estimators` = number of trees. • `max_depth` = tree complexity. • `learning_rate` = learning speed. • `subsample` = row sampling. • `colsample_bytree` = feature sampling.

12. Model Training

`model.fit(X_train_smote, y_train_smote)` Why it is used: Training discovers relationships between features and injuries. The model adjusts thousands of parameters internally to minimize prediction error.

13. Making Predictions

`predict()` returns class labels. `predict_proba()` returns probabilities. Why probabilities matter: A player with 70% injury risk is more informative than simply labeling them as 0 or 1.

14. Evaluation Metrics

Accuracy: Percentage of correct predictions. Precision: Among predicted injuries, how many were correct? Recall: Among actual injuries, how many were detected? F1 Score: Balances precision and recall. AUC-ROC: Measures ranking quality across all thresholds. For imbalanced datasets, AUC and Recall are especially important.

15. Classification Report

The classification report combines: • Precision • Recall • F1 Score • Support Why it is used: Provides detailed insight for each class rather than a single overall metric.

16. Confusion Matrix

Shows: • True Positives • False Positives • True Negatives • False Negatives Why it is used: Allows direct inspection of model mistakes. This is often the first visualization examined after training.

17. ROC Curve

Plots: True Positive Rate vs False Positive Rate. Why it is used: Evaluates model performance at all classification thresholds. AUC near: • 0.5 = random guessing • 1.0 = perfect model

18. Feature Importance

XGBoost calculates how useful each feature was during tree construction. Why it is used: • Explains model behavior. • Identifies injury drivers. • Helps with feature engineering. Higher importance means a feature contributed more to predictions.

19. Top 20 Feature Plot

The bar chart visualizes the most influential features. Why it is used: Visualizations are easier to interpret than raw numbers. This helps identify which player characteristics most influence injury risk.

20. Final Prediction Example

A sample player's statistics are passed into the model. The model outputs: • Injury risk class • Injury probability Why it is used: Demonstrates how the trained model can be applied in a real-world scenario.

21. Complete Pipeline Summary

Pipeline Flow: 1. Install libraries 2. Load dataset 3. Create target variable 4. Select features 5. Handle missing values 6. Split data 7. Apply SMOTE 8. Train XGBoost 9. Evaluate model 10. Explain feature importance 11. Predict future injury risk This workflow represents a complete end-to-end machine learning project.